Power Side-Channel Attacks on Machine-Learning-based Quantum Error Correction Decoders

Anthony Etim
Yale University
New Haven, Connecticut, USA
anthony.etim@yale.edu

Jakub Szefer
Northwestern University
Evanston, Illinois, USA
jakub.szefer@northwestern.edu

Abstract

Machine learning (ML) decoders are increasingly used in quantum error correction (QEC) to model noise, analyze imperfect syndrome measurements, and replace iterative matching algorithms with a learned inference pass, making them a critical component of the classical control stack of a fault-tolerant quantum computer. Prior security analysis of the ML decoders in the controller stack has addressed integrity and fault-injection attacks on the decoders. This work is the first to explore physical power side-channel attacks on ML decoders. In particular, it studies a convolutional Deep Q-Learning decoder for the distance-5 surface code. This work profiles the decoder to capture the power signatures generated during classification tasks and trains a classifier to identify which qubit the decoder identified as having errors based on the power signatures. The trained classifier is then applied to test data, where it recovers the correct data qubit label from the decoder's side-channel data with up to 83.2% accuracy across 25 classes, against 4% for random guessing. A second classifier is trained to identify the type of error labeled by the decoder, Pauli X or Z, and reaches 84.6% accuracy, against 50% for random guessing. This work further performs an initial study of a timing channel, which complements the power side channels, exploring how the duration of the decoder's first convolutional layer can leak the code size for which the decoder is compiled. All the side-channel attacks demonstrated in this work motivate treating the embedded QEC decoder as a confidentiality-critical component of the quantum control stack.

CCS Concepts

• **Security and privacy** → **Side-channel analysis and countermeasures**; *Embedded systems security*; • **Computing methodologies** → *Machine learning*.

Keywords

Power side channels, Machine learning, Quantum error correction

ACM Reference Format:
Anthony Etim and Jakub Szefer. 2026. Power Side-Channel Attacks on Machine-Learning-based Quantum Error Correction Decoders. In *International Workshop on Hardware and Architectural Support for Security and Privacy 2026 (HASP '26), October 31–November 01, 2026, Athens, Greece.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3857307.3857309

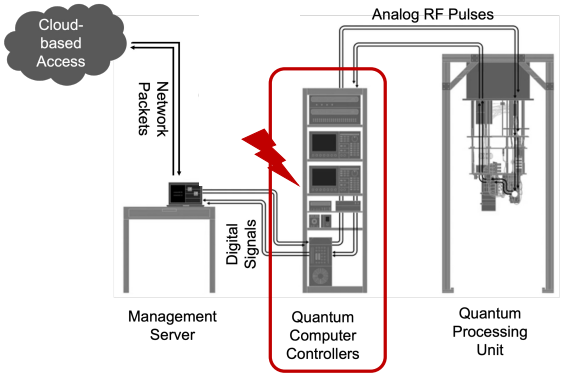


Figure 1: The attack surface in the quantum computer controller. This work assumes physical access and the ability of an attacker to passively monitor decoder operation within the quantum computer controller's classical logic. Previous work mainly focused on actively injecting faults into the decoder.

1 Introduction

Quantum error correction (QEC) relies on a classical decoder that maps measured syndromes to corrective operations. Because decoder decisions affect logical qubits' reliability, the controller running the decoder belongs to the trusted computing base of the quantum system. Decoders based on machine learning (ML) are of growing interest because they can learn the noise characteristics and realize decoding as an inference pass [3, 7, 11, 20]. Their expanded use within quantum computer controllers places the ML-based decoders at a security-critical point of the quantum stack.

Previous work in this area [6] evaluates active fault injection against embedded ML components in the control stack, including faulting the Deep Q-Learning decoder used in this work. That prior study concerns *integrity*: changing operation and results of the decoder through fault injection. We ask the complementary *confidentiality* question: *what information, especially decoder results, can leak through side-channels to outside observers or attackers who have passive access to the quantum controller?* Figure 1 shows the attack surface in the quantum computer controller and the two complementary threats. Active fault injection and passive side-channels both target the controller electronics exposed outside of the quantum processing unit (QPU). Importantly, output correctness, i.e., the absence of fault-injection attacks, alone does not imply that the physical execution is confidential. An attacker who steals information through side-channels can learn which qubits are faulty or which types of errors occurred. Further, side-channel attacks are stealthy as the user or victim does not know the attack is happening.

This work is licensed under a Creative Commons Attribution 4.0 International License.
HASP '26, Athens, Greece
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2964-5/2026/10
https://doi.org/10.1145/3857307.3857309

Table 1: Related work on side-channels in the quantum computing stack.

Work	Observed signal	Inferred information
Erata et al. [5], Xu et al. [25]	Controller power	Control pulses and quantum circuits
TraceQ [22]	Surface-code patch activity	Fault-tolerant circuit dataflow
Shukla et al. [18]	Syndrome stream	Logical-gate fingerprints
This work	ML decoder power	Qubit error location, error type

Within the quantum computing stack, various side-channels have recently been studied, summarized in Table 1.

Among existing side channels, our work is the first to demonstrate side-channel attacks in the ML decoder. The work focuses on multiple aspects of the decoding. First, side channels are used to identify which qubit experienced an error. By training on power side-channel data obtained during profiling, we develop a model that learns the power side-channel signature correlation to the decoder's decision about which qubit experienced an error, for both bit-flip errors and for depolarizing errors. Second, we train a separate model on the power side-channel data obtained during profiling for depolarizing errors and learn the side-channel patterns that map the power traces to the type of depolarizing error, Pauli X or Z, output from the decoder. Third, we study in depth how the power side-channel information is leaked from different layers within the Deep Q-Learning model of the decoder. We focus on CONV1, the first convolutional computation, and FC3, the final dense computation, within the Deep Q-Learning model. Lastly, we perform an initial study of a complementary timing channel.

1.1 Contributions

The contributions of this work are as follows:

- First physical power side-channel evaluation of an ML-based QEC decoder on a commodity side-channel testing platform. Our profiled attack reaches up to 83.2% qubit-label accuracy and 84.6% Pauli X or Z error-type accuracy.
- Evaluation of side-channel leakage from different layers of the Deep Q-Learning model used in the decoder. The side-channel accuracy from FC3-layer power samples exceeds that from CONV1 by 12.1 percentage points for bit-flip errors and 21.5 for depolarizing errors.
- Extension of the side channels to timing channels, with focus on the timing of the CONV1 layer. The CONV1 duration reveals which code size, $d = 5, 7, 9$.

2 Background

2.1 QEC and Syndromes

QEC encodes logical information across multiple physical qubits and repeatedly measures stabilizers to obtain a syndrome. A classical decoder consumes a syndrome history and selects a corrective operation by locating the qubit which experienced an error. A distance-d rotated surface code has d^2 data qubits arranged on a $d \times d$ lattice, together with $d^2 - 1$ ancilla qubits used to measure the stabilizers (syndromes), for a total of $2d^2 - 1$ physical qubits. The syndromes are processed by either conventional matching-based decoding or newer ML-based decoders. ML-based decoders can model richer correlations or replace iterative decoding with a learned forward pass [1, 7, 11, 20]. For the different types of decoders, the key job is to output the label, or location, of the data qubit where an error was detected.

A syndrome does not identify a unique physical error. Any two Pauli errors that differ by a stabilizer produce the same syndrome and act identically on the code space, and errors that differ by a logical operator can also share a syndrome. Consequently, a qubit label is not, in general, a property that an observer of the syndrome (or of the decoder) can recover. We therefore focus on a narrower goal. We assume the adversary first profiles the decoder on a set of training inputs, each corresponding to a single injected error with a known qubit label. Given a new power trace of one of these profiled inputs, not used during training, we aim to recover the associated qubit label. We evaluate the Pauli type (X or Z) separately with a binary classifier.

2.2 Deep Q-Learning Decoder

A Deep Q-Learning decoder is a decoder trained with reinforcement learning. In the decoder of Sweke et al. [20], the state s encodes the current syndrome together with the history of corrections already applied. A convolutional neural network (CNN) maps s to an action value $Q(s, a)$ for each action $a \in \mathcal{A}$, where an action selects the qubit to correct next. The original Python software uses a dueling network, which estimates a state value $V(s)$ and one advantage $A(s, a)$ per action. We port the code to C to use it with the power side-channel measurement setup. Our C port materializes these as one value operand and $|\mathcal{A}|$ advantage operands, which are combined using the standard mean-centered aggregation [23]:

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'). \tag{1}$$

The resulting $|\mathcal{A}|$ Q-values are passed to an arg max that selects the next action. The additional value output is part of the computation we instrument, but it is not shown in the network diagram of Sweke et al.

For $d = 5$, the bit-flip (X) action space contains 25 single-qubit X corrections plus a request-new-syndrome action; the depolarizing (DP) action space adds 25 Z corrections. A direct Y action is unnecessary because a Y error can be corrected using X and Z flips [20].

2.3 Physical Side Channels

CMOS switching activity depends on data and operations. A synchronized power or electromagnetic (EM) measurement can therefore reveal values not intentionally returned by software [10, 14]. In a profiled attack, labeled measurements from a controlled setting train a model applied to later observations. Embedded neural networks are plausible targets because their inference schedules are regular and data-dependent [2, 9, 24]. Here, the target of inference is not a network weight but a label attached to the syndrome information the decoder processes and outputs.

In addition to power, execution duration can disclose configuration information about the target algorithm even when fine-grained analog samples are unavailable. In our C port, the number of serialized CONV1 output positions is $64d^2$. Holding kernels and the action space fixed while changing the code size d therefore changes total work that the decoder has to do. This dependence is not a universal property of QEC decoding, since constant-shape padding, parallelism, or a different accelerator can alter or eliminate it, but it can lead to information leaks via timing.

3 Threat Model

3.1 Adversary's Capabilities

We consider a passive adversary with physical access to the classical controller running the decoding algorithm, shown in Figure 1. We assume the attacker can measure the power consumption of the microcontroller performing the decoding operations. Other physical side-channels could be possible, such as EM, although EM side-channel attacks are not evaluated in this work. We assume the attacker does not glitch the supply, does not modify firmware or weights in the ML model, and does not intentionally perturb inference. This distinguishes our confidentiality attacks from active integrity attacks from previous work [6].

3.2 Profiling and Online Phases

The profiling phase assumes access to the target firmware of the decoder, the finite input set of the syndrome values, the action-to-label mapping used by the decoder, and labeled power trace captures from the target physical controller running the known firmware. This is used to establish the correlation between the power traces and the information the attacker will later want to learn, e.g., which qubit was determined to have an error. This attack assumes the attacker trains on the same device as the actual attack. Training on one physical device while attacking a different one is left for future work.

The online phase classifies power trace captures. The power trace data is split as follows. For each input syndrome, traces 0–29 are used for training (the profiling phase), and traces 30–39 are used for testing. In total 40 power traces are collected for each input syndrome.

3.3 Adversary's Goals

The adversaries we are worried about include insiders, integration or maintenance personnel, and supply-chain actors with physical access to the controller and the decoder. We evaluate three goals for such adversaries:

(1) **Qubit-label localization:** which of the 25 data-qubit labels is associated with an input syndrome being currently processed by the decoder?
(2) **Pauli-type discrimination:** for the depolarizing noise case, is the input syndrome's associated label X or Z?
(3) **Code-size dependent timing:** how do the execution durations change across decoder builds compiled for different code sizes $d \in \{5, 7, 9\}$?

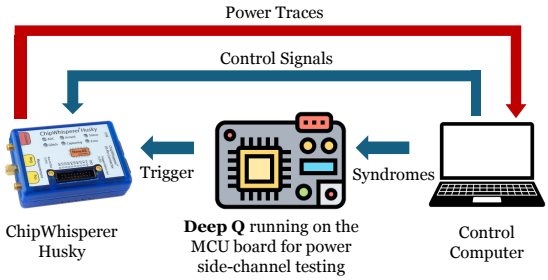


Figure 2: Evaluation setup. The host sends each input (a syndrome volume, or FC3's input vector) to the Cortex-M4 target; the trigger's rising edge starts a fixed-length power-trace capture on the ChipWhisperer Husky, which also counts for how many samples the trigger stays high.

4 Evaluation Setup

We use the Whisp Cloud online service [15] to perform physical power side-channel experiments on the ChipWhisperer Husky boards it provides. We ported the two instrumented layers of the Deep Q-Learning decoder, CONV1 and FC3 (Section 4.1), from the original Python implementation [19, 20] to C, so that they run on the microcontroller target board attached to the ChipWhisperer Husky. The layers in between run in a Python reference model on the host, which supplies FC3's input. The firmware exposes each layer as a separate command. For a CONV1 capture, the host sends a syndrome volume and its action-history masks; for an FC3 capture, it sends the FC3 input vector that the reference model computes for the same volume. Inside the layer, the firmware sets a GPIO trigger line high immediately before one unit of computation starts and sets it low as soon as that unit's result is stored. The unit is the first CONV1 output position or the first FC3 output neuron (Section 5.1). The trigger's rising edge starts a fixed-length power capture on the Husky, which also counts for how many samples the trigger stays high. Repeating this for every input syndrome builds the side-channel trace dataset used for training and testing. For the timing experiment, a separate firmware build sets the trigger high before the first CONV1 output position and low after the last one, so the trigger-high count measures the duration of the whole layer. Figure 2 shows a simplified diagram of the setup.

4.1 Decoder, Layers, and Embedded Target

We use the Deep Q-Learning decoder of Sweke et al. [20] and its X-error (bit-flip) and depolarizing (DP) variants. The Deep Q-Learning network consumes stacks of spatial syndrome and history matrices, and its dueling head produces action scores using Eq. 1. Within the decoder's code (equivalently, its neural-network layers), we focus on two distinct regions:

- **CONV1**, the first convolutional computation, is the first layer that processes the raw spatial representation. In our C port, it computes $64d^2$ output positions serially (64 filters at each of $d \times d$ positions), whose values feed the next layer.
- **FC3**, the final dense computation, emits the value and advantage operands used to construct action scores. In our C port, it maps an $|\mathcal{A}|$-dimensional input to these $|\mathcal{A}| + 1$ outputs.

Table 2: Traces per task in each capture session. Each input is captured 40 times; training uses repeat indices numbered 0–29 and testing uses 30–39.

Task	Inputs	Classes	Train	Test
X qubit localization	25	25	750	250
DP qubit localization	50	25	1500	500
DP Pauli X/Z	50	2	1500	500

Our C port runs on a CW312T-SAM4S target (ATSAM4S2A, Cortex-M4) connected to a ChipWhisperer Husky [16] accessed via the Whisp Cloud online service [15]. This platform provides a dedicated power-measurement path and hardware triggering, all done remotely for ease of experimentation.

4.2 Dataset, Actions, and Classification Tasks

The dataset, stored in NumPy's `.npz` format (a compressed archive of arrays), contains test inputs for the two neural-network decoders of Sweke et al. [20]. Each input is a description of the code's current state that a decoder takes in before choosing an action. One decoder was trained on bit-flip noise (X model) and the other on depolarizing noise (DP model), both for the distance-5 surface code with $5{\times}5 = 25$ data qubits. Each input is a binary tensor of shape $5 \times 6 \times 6$, i.e., a stack of five 6×6 slices, where each slice holds the code's stabilizer outcomes on a 6×6 grid. The action-history masks sent with each input are empty, since no correction has yet been applied. After removing duplicates, the files contain 26 distinct inputs for the X model and 51 for the DP model. These numbers equal the sizes of the two action spaces, and each distinct input corresponds to exactly one action.

In the X model, actions 0–24 apply an X correction to one of the 25 data qubits, and action 25 is a terminal action that applies no correction. The DP model has 51 actions: 0–24 apply X corrections, 25–49 apply Z corrections, and 50 is the same terminal action. The export labels the terminal action `I` with the placeholder coordinate $(-1, -1)$, and our reference pipeline calls it `IDLE/STOP`. Rather than acting on a data qubit, it ends the current round of corrections and requests a fresh syndrome measurement [20].

We define two classification tasks in which each input is labeled by the correction it corresponds to. Because the terminal action has no data-qubit coordinate and no Pauli type, we exclude it from both tasks. This leaves 25 X-model inputs (one per qubit) and 50 DP-model inputs (one X and one Z input per qubit), so every class contains the same number of examples and both tasks are balanced. Table 2 lists the resulting number of traces per task in each capture session, in which each distinct input is captured 40 times. These repeats do not add new information: each task contains only 25 or 50 distinct inputs.

5 Attack Implementation

5.1 Trace Capture

Each round, every stabilizer of the surface code is measured, giving one bit per stabilizer: 0 if the check is satisfied, 1 if it detects an error nearby. These stabilizers sit at fixed positions on the code's lattice. For a distance-5 code, that grid fits in a 6×6 matrix, with each cell holding one stabilizer outcome (or 0 where there is no stabilizer). That is one slice. A syndrome volume is a set of slices, one for each time step. The decoder looks at multiple time steps to determine the corrective action, hence the input is one volume of syndromes.

To capture traces, a volume of syndromes is sent to the firmware running on the microcontroller (for FC3, the corresponding FC3 input; Section 4). The microcontroller runs at approximately 7.36 MHz target clock, with eight power samples per clock, corresponding to about 58.9 MHz sampling. The ChipWhisperer Husky's trigger counter (`trig_count`) also counts in samples, so all durations below are given in samples; one target clock cycle is eight samples. We collect 131,000 samples for the CONV1 layer and 24,000 samples for the FC3 layer during each capture. The two capture windows differ in how much of their target computation they cover.

For CONV1, the trigger covers the first of the 1,600 output positions ($64d^2$ at $d = 5$) and is high for 15,012 samples on X and 16,308 on DP on average, but the recorded window is 131,000 samples long, 8.7× and 8.0× the computation the trigger covers. We set the CONV1 window close to the Husky's 131,070-sample maximum for a single non-streaming capture, to record as much of the layer as one capture allows; the whole layer lasts 20.9 million samples on X at $d = 5$ (Table 4). The firmware keeps computing CONV1 after the trigger falls, so the window holds the triggered position followed by roughly eight to nine further output positions of the same filter.

For FC3, the window is much closer in length to the computation the trigger covers. The trigger covers the first output neuron, which computes the state value $V(s)$ (the outputs are ordered $V(s)$ followed by the $|\mathcal{A}|$ advantages), and its dot product has length $|\mathcal{A}|$. The measured `trig_count` is 23,882 samples for X and 46,175 for DP. Their ratio, 1.93, closely follows the ratio of action counts, $51/26 = 1.96$, and corresponds to a consistent cost of 905–919 samples (about 113–115 target clock cycles) per multiply-accumulate in both builds, as the firmware uses software floating point. Because the FC3 window records 24,000 samples, it captures a complete X neuron in 96.2% of captures (in the remaining 3.8% the computation runs slightly longer than the window) but only the first 52% of a DP neuron.

For the evaluation we do not focus on the specific length of each capture window. Instead, we compare how much information can be extracted from each window. We neither normalize leakage per operation nor rank complete layers by leakage. The power side-channel classification uses only the two capture windows (131,000 samples for CONV1 and 24,000 for FC3), and the timing analysis uses only the scalar `trig_count`.

The timing analysis uses a different trigger from the power captures. A single CONV1 output position does the same work at every d, so it cannot reveal the lattice size; the timing builds instead hold the trigger high across the whole CONV1 layer (Section 4). This is why the $d = 5$ X duration in Table 4 is roughly 1,400 times the 15,012-sample single-position trigger above. The factor is smaller than the 1,600-position count because CONV1 skips the accumulation for zero-valued inputs, so the cost of a position depends on its inputs: averaged over the layer, a $d = 5$ X position takes 13,067 samples.

5.2 Train/Test Split and Preprocessing

The decoder is deterministic, so running it once on an input produces one trace. Each trace, however, contains noise and variations due to changing environment. We perform repeated captures to get multiple traces for each input. For each input, captures (also called repeats) are numbered 0–39; repeats 0–29 are used for training and repeats 30–39 for testing. Table 2 shows the number of traces captured for each type of experiment. The experiments measure whether for a given input known to the attacker (used in training) the same input can be recognized from the testing traces. The experiments therefore measures whether the classifier recognizes new captures of inputs it has already seen.

For processing, each trace is reduced by averaging consecutive blocks of 100 samples, turning 131,000 CONV1 samples into 1,310 features and 24,000 FC3 samples into 240 features. The features are then standardized using the mean and variance of the training set only, and the same transformation is applied to the test set. CONV1 and FC3 traces go through this same pipeline.

5.3 Power Side-Channel Classifier

To classify the power traces, we developed our own classifier. The classifier is a multilayer perceptron with two hidden layers using ReLU activations (100 and 50 units for the 25-class tasks, 64 and 32 for the binary task), trained with the Adam optimizer and early stopping. We use the full window as input and do not select points of interest or otherwise use knowledge of where leakage is expected to occur.

Each task and window is evaluated in three capture sessions, and in each session the classifier is trained with five random seeds, giving 15 trained models in total. We report the mean accuracy over all 15. We report two measures of variability. Session variability is the standard deviation of the three per-session mean accuracies, computed with Bessel's correction $(n-1)$ [4]. Seed variability is computed by first taking, within each session, the standard deviation over the five seeds without Bessel's correction (n), and then averaging these three values.

5.4 Timing Side-Channel Measurement

In addition to power side channels, we perform an initial timing-channel study. To measure how execution time depends on code size, the timing experiments compile the firmware for code distances $d \in \{5, 7, 9\}$. The convolution kernels are kept at the same size, but the implementation computes outputs one at a time, and the number of outputs grows as $64d^2$. The network is not retrained for larger d. All three builds reuse the trained $d = 5$ weights and the same action space, and each $d = 5$ input is placed in the upper-left corner of a larger lattice padded with zeros. The experiment therefore measures how execution time grows with the size of the convolution. It does not test a decoder that actually operates at $d = 7$ or $d = 9$. These builds use the whole-layer trigger (Section 4), and for each build and model we record 1,000 captures, cycling through the dataset inputs. The timing experiments require no classifier: the measured durations are compared directly to find thresholds that separate the code sizes.

Table 3: Power-classification aggregates. "Acc." (Accuracy) is the mean over three capture sessions and five classifier seeds per session. "Sess." (Session) and "Seed" are the two standard deviation components described in Section 5.3.

Task	Window	Acc.	Sess.	Seed
Qubit loc., X (25)	CONV1	.512	.016	.011
	FC3	.633	.020	.032
Qubit loc., DP (25)	CONV1	.617	.008	.008
	FC3	.832	.026	.019
Pauli X/Z, DP (2)	CONV1	.830	.007	.012
	FC3	.846	.023	.029

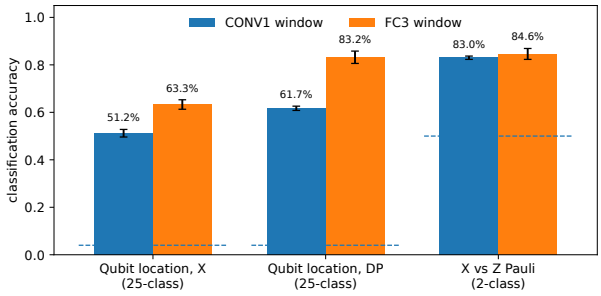


Figure 3: Aggregates from Table 3. Dashed lines are uniform-random baselines implied by the class counts (4% for localization and 50% for X/Z); error bars are the standard deviation of session means.

6 Results

This section presents the results of our work. It follows the three goals we set to explore. (1) Qubit-label localization: which of the 25 data-qubit labels is associated with an input syndrome being currently processed by the decoder? (2) Pauli-type discrimination: for the depolarizing noise case, is the input syndrome's associated label X or Z? (3) Code-size dependent timing: how do the execution durations change across decoder builds compiled for different code sizes $d \in \{5, 7, 9\}$?

6.1 Recovering the Qubit Label

For qubit localization, we have 25 classes and a uniform random predictor has 4% expected accuracy. For the X model, the classifier reaches 51.2% from CONV1-window traces and 63.3% from FC3-window traces; for the DP model, it reaches 61.7% and 83.2%, respectively. Table 3 lists these aggregates with their variability, and Figure 3 plots them. FC3 accuracy exceeds CONV1 accuracy by 12.1 percentage points for X and 21.5 for DP. These accuracies are also far above random guessing, indicated by dashed lines in the figure, and indicate substantial information extracted from the traces.

Note that the classifier identifies the label only from among the profiled inputs. For X, each qubit label has one exact input volume. For DP, each qubit has two volumes, one associated with X and one with Z. The same volumes are used in the training and testing; they are used to generate the power traces, of which 75% is used for training and 25% for testing, giving the 30/10 split. The

Table 4: CONV1 layer trigger-high durations for three different codes sizes d. "Mean" is the mean duration computed from $1,000$ captures each, counted in samples (eight per target clock cycle). "Std." is the standard deviation.

Model	d	Mean	Std.
X	5	20,906,513	459,057
	7	35,805,028	482,546
	9	60,321,451	469,593
DP	5	23,221,171	564,235
	7	39,796,318	670,362
	9	67,414,744	652,466

side-channel attack cannot currently identify the qubit labels for syndrome sequences (i.e., volumes) not used in training.

As a further analysis, Figure 4 shows confusion matrices for the DP localization output from one classifier seed and capture session for the two layers of the decoder's neural network. Future work can explore why certain qubit locations (i.e., indices) are better identified while true qubit and predicted qubit locations vary for certain qubits quite a bit.

6.2 Recovering the Pauli Type

For Pauli-type determination, we have two classes, X and Z, each with 25 inputs (one per qubit), and a uniform random predictor has 50% expected accuracy. For the DP model, the classifier reaches 83.0% from CONV1-window traces and 84.6% from FC3-window traces. Figure 3 plots these aggregates and it can be observed that the results are much better than random guessing.

6.3 Leaking Code Size Through Timing

Serialized CONV1 work grows as $64d^2$, and the measured means increase strongly with the code size d, as Figure 5 shows. Adjacent means differ by 14.9 and 24.5 million samples for X, and by 16.6 and 27.6 million for DP. Every adjacent difference is at least 24.7× the larger standard deviation of its pair. The per-code size ranges are also disjoint: the longest capture of each build is at least 13.3 million samples shorter than the shortest capture of the next larger build. The timing is deterministic: two further capture sessions reproduced every per-capture count exactly, so the spread in Table 4 comes from input-dependent work, not measurement noise. An observer who can measure the CONV1 interval can therefore identify the code size d.

7 Discussion

7.1 Comparison of Side Channels from Different Layers

Which layer of the decoder's neural network leaks more depends on the type of information the attacker wants to target. FC3 layer accuracy is substantially higher for qubit localization under the evaluated observations, while the 1.6-point Pauli-type gap is small relative to the measured variability. A plausible explanation is what the final dense computation does: it produces the operands from which correction scores are formed. CONV1 instead transforms a spatial syndrome/history tensor. The current data do not isolate that architectural explanation, because window length, instrumentation position, signal-to-noise ratio, and exposed work also differ. A controlled follow-up should compare equal-time observations, equal multiply-accumulate counts, several CONV1 positions, and whole-layer captures where hardware permits.

For learning the code size, the measured CONV1 duration changes with the serialized position count while FC3's action space is held fixed. These observations do not establish a universal ranking of leakage per operation or per complete layer, but suggest attacker can target different machine learning layers to extract different information.

7.2 Security Implications of the Side Channel

Our power side channel attacks show that power consumption during execution of the ML decoders execution contains measurable information about labels attached to profiled inputs. This complements fault injection, since defenses that check only wrong outputs, resets, non-finite values, or redundant-inference disagreement do not address passive measurement at all.

It should be noted that the attacks do not reconstruct a unique arbitrary physical error. QEC syndromes are degenerate, localization and type are separate classifiers, and every exact input volume is represented during profiling. Future work can explore how to expand the classification to unknown inputs.

7.3 Implementation and Device Dependence

The measured timing dependence is consistent with direct serialized convolution geometry. Increasing the compiled geometry from $d = 5$ to $d = 9$ multiplies the CONV1 position count by $(9/5)^2 \approx 3.24$, while the mean duration grows by 2.89× (X) and 2.90× (DP), because the cost of each position depends on its inputs (Section 5.1). A constant-shape design that equalizes executed work could reduce this position-count timing distinction, but we have not evaluated one. Parallel FPGA or RFSoC implementations may transform the effect into a different duration, bandwidth, or activity pattern. We have not measured those platforms and they are left as future work.

We also run CONV1 and FC3 as separate commands, with the layers in between on the host; a fully embedded decoder that runs all layers back to back is not evaluated. All classifier training and testing use one physical Cortex-M4 target. Process variation and acquisition-path differences can shift trace distributions across devices. The numbers therefore support same-device recognition under our capture conditions only. Each session's classifier is also trained and tested on that session's own captures, and the sessions were recorded close together, so transfer across sessions and over longer time gaps is untested. Multiple boards, probes, gains, time gaps, and environmental conditions are needed to assess portability and calibration cost.

8 Related Work

Learned decoders for QEC. Decoding has been approached with reinforcement learning on the toric and surface codes [1, 7, 20], with graph neural networks over detector graphs [11], and with recurrent transformer decoders that beat matching on hardware data [3]. That work optimizes accuracy, latency, and noise modeling,

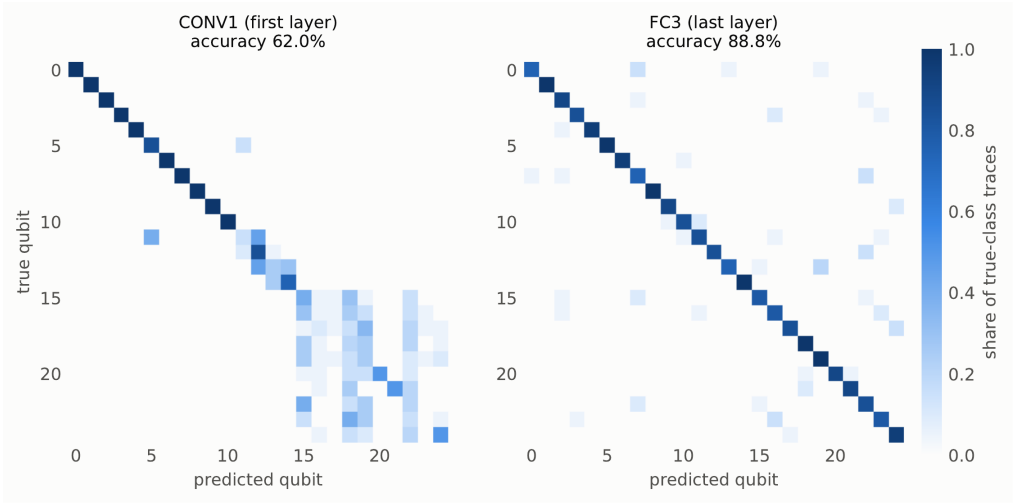


Figure 4: Single-run DP localization confusion matrices. The displayed 62.0% (CONV1) and 88.8% (FC3) are single-run values; Table 3 reports the corresponding means of 61.7% and 83.2%.

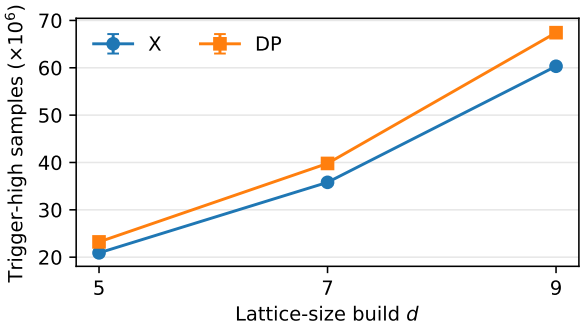


Figure 5: Duration of the CONV1 layer as a function of the code size; means and standard deviations plotted from the data in Table 4.

and evaluates decoders by logical error rate. None of it addresses the security of the decoder implementation.

Physical leakage from neural networks. Batina et al. recover neural-network architecture from EM emanations, Hua et al. infer CNN structure through side-channel information, Wei et al. recover input images from accelerator power, and DeepEM recovers model information from EM traces [2, 9, 24, 28]. Profiled machine learning is also widely used as an attack method in physical side-channel analysis [13, 17]. These works establish ML inference as a leakage surface, but do not study an ML decoder processing QEC syndromes.

Quantum-computing side channels and attacks. Quantum-controller power has been analyzed for control-pulse and circuit reconstruction [5, 25], while cloud timing can disclose properties of remotely executed jobs [12]. TraceQ reconstructs fault-tolerant circuit dataflow from surface-code patch-activity traces [22]. Other work studies QSVT leakage, pulse-level attacks, fault injection, and broader quantum-hardware security [8, 21, 26, 27]. Recent decoder work exploits gate fingerprints in syndrome streams [18]. These channels observe control pulses, service timing, patch activity, or syndrome structure. We instead measure the physical execution of the classical processor running an ML QEC decoder. Table 1 summarizes these closest distinctions.

Integrity of the same control path. The closest is a previous active fault-injection study of the same Deep Q-Learning implementation and measurement platform [6]. It perturbs CONV1 and the final dueling-output computation to change decoder actions. We leave the computation unglitched and study confidentiality. To our knowledge, no prior work reports a physical power side-channel evaluation of an embedded ML-based QEC decoder.

9 Conclusion

This work presented the first physical power and timing side-channel study of an embedded ML-based QEC decoder, using a Deep Q-Learning decoder for the distance-5 surface code on a Cortex-M4 target. Using C port of decoder's code instrumented with trigger points, we collected power traces for CONV1 and FC3 layers of the ML model. We then developed a classifier that recovers the data-qubit label with up to 83.2% accuracy across 25 classes and the Pauli type with 84.6%. We showed that the final dense layer leaks more than the first convolutional layer for localization while for Pauli type we measure no advantage for either layer. For timing attack, we showed that mean CONV1 duration rises sharply as code size d increases. These results motivate treating embedded QEC decoder inference as confidentiality-critical as attackers who have access to the physical controller can easily collect power traces and find out the critical information from decoder's operation.

Acknowledgments

This work was supported in part by National Science Foundation grants 2245344 and 2332406. We would like to acknowledge the free access and use of the Whisp Cloud online service [15] for running the power side-channel data collection.

References

[1] Philip Andreasson, Joel Johansson, Simon Liljestrand, and Mats Granath. 2019. Quantum error correction for the toric code using deep reinforcement learning. *Quantum* 3 (2019), 183. https://doi.org/10.22331/q-2019-09-02-183

[2] Lejla Batina, Shivam Bhasin, Dirmanto Jap, and Stjepan Picek. 2019. CSI NN: Reverse engineering of neural network architectures through electromagnetic side channel. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, Santa Clara, CA, 515–532.

[3] Johannes Bausch, Andrew W. Senior, Francisco J. H. Heras, Thomas Edlich, Alex Davies, Michael Newman, Cody Jones, Kevin Satzinger, Murphy Yuezhen Niu, Sam Blackwell, George Holland, Dvir Kafri, Juan Atalaya, Craig Gidney, Demis Hassabis, Sergio Boixo, Hartmut Neven, and Pushmeet Kohli. 2024. Learning high-accuracy error decoding for quantum processors. *Nature* 635, 8040 (2024), 834–840. https://doi.org/10.1038/s41586-024-08148-8

[4] George Casella and Roger L. Berger. 2002. *Statistical Inference* (2nd ed.). Duxbury, Pacific Grove, CA.

[5] Ferhat Erata, Chuanqi Xu, Ruzica Piskac, and Jakub Szefer. 2024. Quantum Circuit Reconstruction from Power Side-Channel Attacks on Quantum Computer Controllers. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2024, 2 (2024), 735–768. https://doi.org/10.46586/tches.v2024.i2.735-768

[6] Anthony Etim and Jakub Szefer. 2026. Fault Attacks on ML-based Quantum Control and Error Correction. arXiv:2512.20077 [quant-ph] https://arxiv.org/abs/2512.20077

[7] David Fitzek, Mattias Eliasson, Anton Frisk Kockum, and Mats Granath. 2020. Deep Q-learning decoder for depolarizing noise on the toric code. *Physical Review Research* 2, 2 (2020), 023230. https://doi.org/10.1103/PhysRevResearch.2.023230

[8] Swaroop Ghosh, Suryansh Upadhyay, and Abdullah Ash Saki. 2025. A primer on security of quantum computing hardware. *Proc. IEEE* 113, 7 (2025), 640–667. https://doi.org/10.1109/JPROC.2025.3630989

[9] Weizhe Hua, Zhiru Zhang, and G. Edward Suh. 2018. Reverse engineering convolutional neural networks through side-channel information leaks. In *55th ACM/ESDA/IEEE Design Automation Conference (DAC)*. Association for Computing Machinery, 1–6. https://doi.org/10.1145/3195970.3196105

[10] Paul Kocher, Joshua Jaffe, and Benjamin Jun. 1999. Differential power analysis. In *Advances in Cryptology — CRYPTO*. Springer, 388–397. https://doi.org/10.1007/3-540-48405-1_25

[11] Moritz Lange, Pontus Havström, Basudha Srivastava, Isak Bengtsson, Valdemar Bergentall, Karl Hammar, Olivia Heuts, Evert van Nieuwenburg, and Mats Granath. 2025. Data-driven decoding of quantum error correcting codes using graph neural networks. *Physical Review Research* 7, 2 (2025), 023181. https://doi.org/10.1103/PhysRevResearch.7.023181

[12] Chao Lu, Esha Telang, Aydin Aysu, and Kanad Basu. 2025. Quantum Leak: Timing Side-Channel Attacks on Cloud-Based Quantum Services. In *Proceedings of the Great Lakes Symposium on VLSI 2025 (GLSVLSI)*. Association for Computing Machinery, 252–257. https://doi.org/10.1145/3716368.3735264

[13] Houssem Maghrebi, Thibault Portigliatti, and Emmanuel Prouff. 2016. Breaking cryptographic implementations using deep learning techniques. In *International Conference on Security, Privacy, and Applied Cryptography Engineering (SPACE)*. Springer, 3–26. https://doi.org/10.1007/978-3-319-49445-6_1

[14] Stefan Mangard, Elisabeth Oswald, and Thomas Popp. 2007. *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. Springer. https://doi.org/10.1007/978-0-387-38162-6

[15] Northwestern University. 2025. Whisp Cloud: On-demand cloud-based access to side-channel analysis and fault-injection tools. Retrieved October 17, 2026 from https://whisp.cloud

[16] Colin O'Flynn and Zhizhang Chen. 2014. ChipWhisperer: An open-source platform for hardware embedded security research. In *International Workshop on Constructive Side-Channel Analysis and Secure Design (COSADE)*. Springer, 243–260. https://doi.org/10.1007/978-3-319-10175-0_17

[17] Stjepan Picek, Guilherme Perin, Luca Mariot, Lichao Wu, and Lejla Batina. 2023. SoK: Deep learning-based physical side-channel analysis. *Comput. Surveys* 55, 11, Article 227 (2023), 35 pages. https://doi.org/10.1145/3569577

[18] Shashvat Shukla, Dan E. Browne, and Shin Nishio. 2026. Anticipating Decoder Side-channel Attacks in Fault-tolerant Quantum Computers. arXiv:2607.12174 [quant-ph]

[19] Ryan Sweke. 2018. DeepQ-Decoding: Decoders for fault tolerant quantum computation via deep Q reinforcement learning. GitHub repository. Retrieved October 17, 2026 from https://github.com/R-Sweke/DeepQ-Decoding

[20] Ryan Sweke, Markus S. Kesselring, Evert P. L. van Nieuwenburg, and Jens Eisert. 2021. Reinforcement learning decoders for fault-tolerant quantum computation. *Machine Learning: Science and Technology* 2, 2 (2021), 025005. https://doi.org/10.1088/2632-2153/abc609

[21] Kidus Tessma, Hrvoje Kukina, and Jakub Szefer. 2025. Recovering QSVT polynomials from side-channel information on quantum computers. In *IEEE 43rd International Conference on Computer Design (ICCD)*. IEEE, 342–347. https://doi.org/10.1109/ICCD65941.2025.00056

[22] Theodoros Trochatos, Christopher Kang, Andrew Wang, Frederic T. Chong, and Jakub Szefer. 2026. TraceQ: Trace-Based Reconstruction of Quantum Circuit Dataflow in Surface-Code Fault-Tolerant Quantum Computing. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–14. https://doi.org/10.1109/HPCA68181.2026.11408545

[23] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. 2016. Dueling Network Architectures for Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 48)*. PMLR, 1995–2003.

[24] Lingxiao Wei, Bo Luo, Yu Li, Yannan Liu, and Qiang Xu. 2018. I know what you see: Power side-channel attack on convolutional neural network accelerators. In *34th Annual Computer Security Applications Conference (ACSAC)*. Association for Computing Machinery, 393–406. https://doi.org/10.1145/3274694.3274696

[25] Chuanqi Xu, Ferhat Erata, and Jakub Szefer. 2023. Exploration of Power Side-Channel Vulnerabilities in Quantum Computer Controllers. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Association for Computing Machinery, New York, NY, USA, 579–593. https://doi.org/10.1145/3576915.3623118

[26] Chuanqi Xu, Ferhat Erata, and Jakub Szefer. 2024. Quantum computer fault injection attacks. In *IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 1. 331–337. https://doi.org/10.1109/QCE60285.2024.00047

[27] Chuanqi Xu and Jakub Szefer. 2025. Security attacks abusing pulse-level quantum circuits. In *IEEE Symposium on Security and Privacy (S&P)*. IEEE, 222–239. https://doi.org/10.1109/SP61157.2025.00083

[28] Honggang Yu, Haocheng Ma, Kaichen Yang, Yiqiang Zhao, and Yier Jin. 2020. DeepEM: Deep neural networks model recovery through EM side-channel information leakage. In *IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 209–218. https://doi.org/10.1109/HOST45689.2020.9300274